

CMSE 801 - Day 7



Plotting data with matplotlib and using NumPy to
manipulate arrays and perform calculations

September 18, 2024

Attention: We have **new** table assignments -- make sure to sit your **new** group!

General Course Announcements

- **Reminder:** Homework 1 is due Friday, September 27.

Questions/comments from Day 7 PCA

- Some issues with getting the ".csv" file loaded into your notebook (e.g. issues with download, file not in the right place, etc.). **We'll try to get these resolved in class today!**
- Some questions about what various arguments in the `np.loadtxt()` function do -
- **we'll take a look at this and break it down.**
- General comments around not quite being used to NumPy arrays yet (how do you modify them, why are the values displayed/stored a certain way, etc) and still feeling like you have to look things up a lot to remember how to use various functions (e.g. when making a matplotlib plot, the new array methods).
- This PCA felt a little long -- **sorry about that!**

Let's try loading up the data from the PCA

For reference, we may want to take a look at the [NumPy documentation](#).

```
In [1]: # Import numpy before I try to use it!
import numpy as np

# Load the data using NumPy's loadtxt function
pop_dates, pop_numbers = np.loadtxt("michigan_pop.csv", usecols = (0,1))
```

What do the following arguments do?

- `usecols` : tells NumPy which columns of the dataset to load
- `skiprows` : tells NumPy if it should skip any of the rows and, if so, how many to skip
- `delimiter` : tells NumPy what character(s) are used to separate columns in the file
- `unpack` : splits the columns into separate variables

Let's take a peek at our data

```
In [2]: # Let's look at just the first 5 values in each of my new variable  
print("First five dates:", pop_dates[0:5])  
print("First five population values:", pop_numbers[0:5])
```

First five dates: [1900. 1901. 1902. 1903. 1904.]

First five population values: [2423000. 2450000. 2477000. 2505000.
0. 2534000.]

```
In [3]: # Or I could have used a loop!  
for i in range(5):  
    print(pop_dates[i], pop_numbers[i])
```

1900.0 2423000.0

1901.0 2450000.0

1902.0 2477000.0

1903.0 2505000.0

1904.0 2534000.0

```
In [4]: # What if I don't use "unpack"
pop_dates, pop_numbers = np.loadtxt("michigan_pop.csv", usecols = (0,1))
```

```
-----
-----
ValueError                                Traceback (most recent
call last)
Cell In[4], line 2
      1 # What if I don't use "unpack"
----> 2 pop_dates, pop_numbers = np.loadtxt("michigan_pop.csv",
      usecols = (0,1), skiprows = 1, delimiter=',')
```

ValueError: too many values to unpack (expected 2)

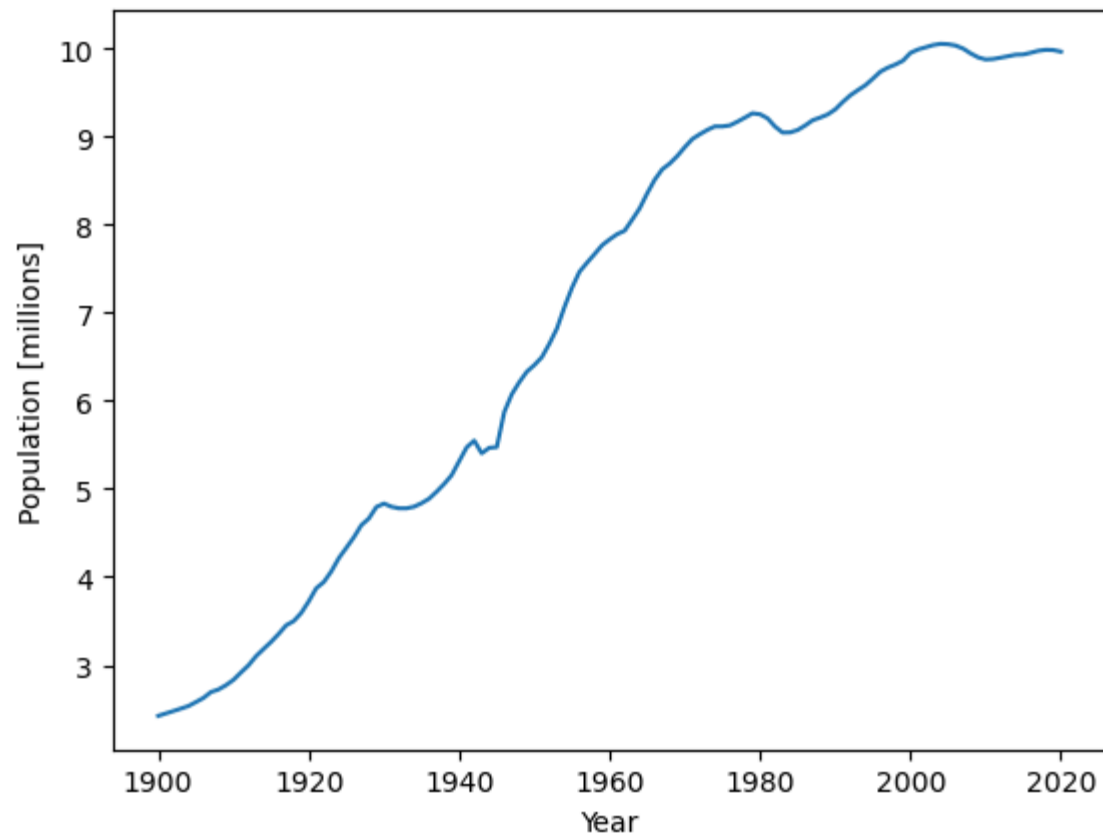
```
In [5]: pop_values = np.loadtxt("michigan_pop.csv", usecols = (0,1), skiprows =
```

```
In [6]: # Do I still have all the same values?
print(pop_values[0:5,:]) # note that for 2D arrays, the first index is
```

```
[[1.900e+03 2.423e+06]
 [1.901e+03 2.450e+06]
 [1.902e+03 2.477e+06]
 [1.903e+03 2.505e+06]
 [1.904e+03 2.534e+06]]
```

Let's take a look at this data since we've gone through the trouble of loading it up

```
In [7]: # Need to import before I can make plots!  
import matplotlib.pyplot as plt  
  
plt.plot(pop_dates, pop_numbers/1e6)  
plt.xlabel('Year')  
plt.ylabel('Population [millions]');
```



What if I needed to add some more values? (important to check the [documentation](#))

```
In [8]: # appending to an array (different than how strings work!)  
# be mindful of the choice to overwrite the original data  
pop_dates = np.append(pop_dates, [2021, 2022, 2023, 2024])  
print(pop_dates)  
  
# And I could do the same thing with the "pop_numbers" array!
```

```
[1900. 1901. 1902. 1903. 1904. 1905. 1906. 1907. 1908. 1909. 191  
0. 1911.  
1912. 1913. 1914. 1915. 1916. 1917. 1918. 1919. 1920. 1921. 192  
2. 1923.  
1924. 1925. 1926. 1927. 1928. 1929. 1930. 1931. 1932. 1933. 193  
4. 1935.  
1936. 1937. 1938. 1939. 1940. 1941. 1942. 1943. 1944. 1945. 194  
6. 1947.  
1948. 1949. 1950. 1951. 1952. 1953. 1954. 1955. 1956. 1957. 195  
8. 1959.  
1960. 1961. 1962. 1963. 1964. 1965. 1966. 1967. 1968. 1969. 197  
0. 1971.  
1972. 1973. 1974. 1975. 1976. 1977. 1978. 1979. 1980. 1981. 198  
2. 1983.  
1984. 1985. 1986. 1987. 1988. 1989. 1990. 1991. 1992. 1993. 199  
4. 1995.  
1996. 1997. 1998. 1999. 2000. 2001. 2002. 2003. 2004. 2005. 200  
6. 2007.  
2008. 2009. 2010. 2011. 2012. 2013. 2014. 2015. 2016. 2017. 201  
8. 2019.  
2020. 2021. 2022. 2023. 2024.]
```

Goals for today

- Load and explore some data using NumPy
- Get used to working with NumPy **arrays**
- Visualize that data using matplotlib
- If there's time, look for correlations in the data

Looking forward - Day 8 PCA: Thinking about how we can modeling dynamics systems by solving ordinary differential equations numerically

- Ordinary differential equations (ODEs) are equations that **provide information about how a system evolves over times.**
- We can use this information to start in the system in a particular state and then move it forward in time *numerically*, i.e. we can approximate how the system should evolve.
- This can be really handy for modeling dynamic systems.

Also, there will be an opportunity to start thinking about what you'd want to work on your for semester project.

